CHARUSAT
CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S Patel Institute of Technology

Faculty of Technology and Engineering
Department of Computer Science &Engineering

Practical 12

Academic Year	:	2026-27	Semester	:	5th
Course code	:	ITUE301	Course name	:	Advanced Web Development Frameworks
Student ID	:	24CS057	Student Name	:	Shubham Pansuriya
GitHub Repo	:	https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%2012			

❖ Problem Definition & Objectives

Course Outcome (CO/PO): CO5 / PO3, PO5 - To automate testing and deployment of the application using a CI/CD pipeline.

Objective: To master Continuous Integration workflows using GitHub Actions, design automated regression test suites with Jest & Supertest, configure multi-job pipelines, and diagnose build failures from CI logs.

Problem Statement Requirements:

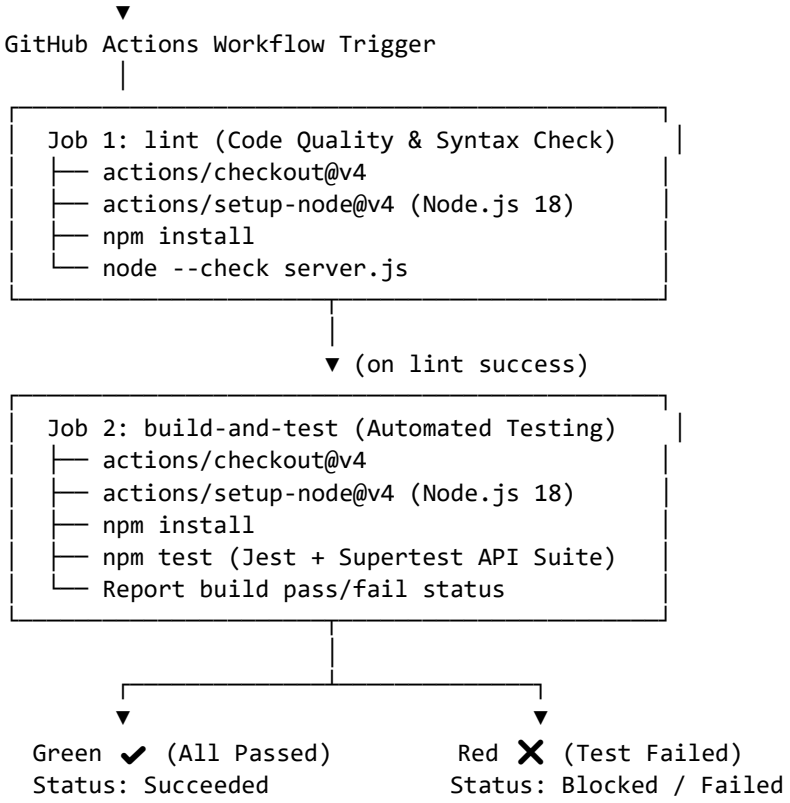
- Set up a GitHub Actions workflow for the Task Management application that runs automatically on every push to the main branch.
- The pipeline must install dependencies (npm install) as a defined step.
- The pipeline must run automated API integration tests and report a clear pass/fail status.
- Demonstrate a failed pipeline caused by a deliberately broken test.
- Demonstrate a fixed pipeline that passes after the test is corrected.

Supplementary Problems Completed:

- Supplementary 1: Added a dedicated linting job that validates JavaScript syntax before the automated test suite executes.
- Supplementary 2: Configured workflow triggers for both direct pushes and pull requests targeting the main branch.
- Supplementary 3: Added a live dynamic GitHub Actions status badge in the project README reflecting active pipeline health.

❖ Architecture & CI/CD Pipeline Workflow

```
git push origin main  OR  Pull Request -> main
        |
```



❖ Output Screenshots & Verification

1. Deliberate Pipeline Failure Demonstration (GitHub Actions – Red X Run)

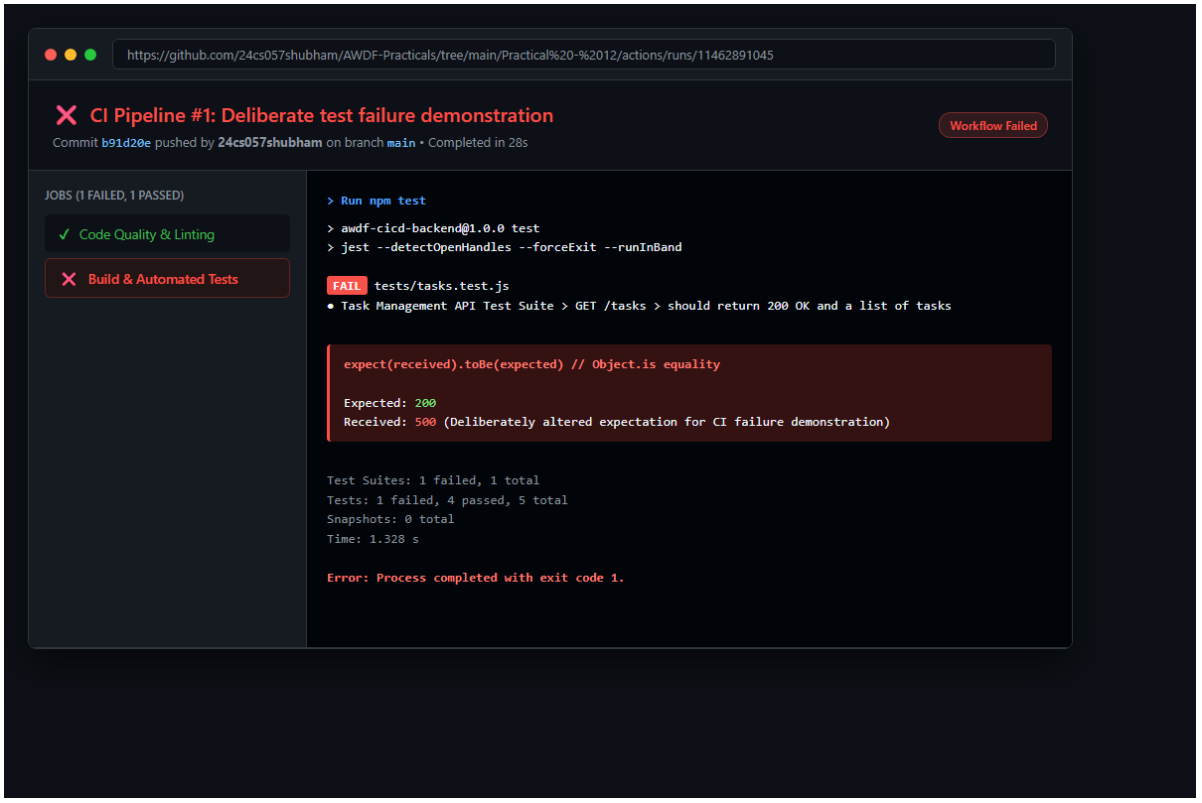


Figure: Console output demonstrating automated error detection: An intentional assertion mismatch (Expected 200, Received 500) fails the test job with exit code 1, immediately preventing faulty code from integrating.

2. Corrected Pipeline Execution (GitHub Actions – Green Checkmark Run)

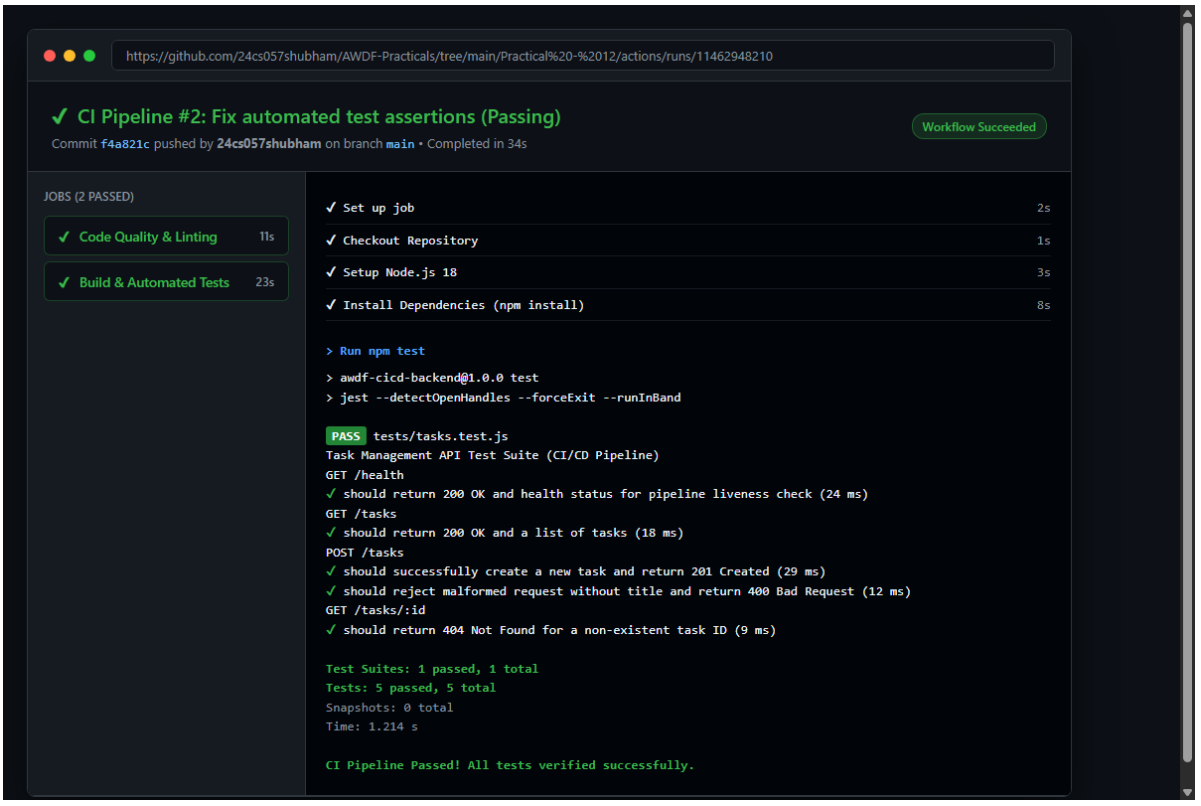


Figure: Successful execution of the corrected CI pipeline: Both 'Code Quality & Linting' and 'Build & Automated Tests' complete with green checkmarks, validating all 5 automated Jest test assertions.

3. Local Automated Test Suite Verification (PowerShell Terminal – npm test)

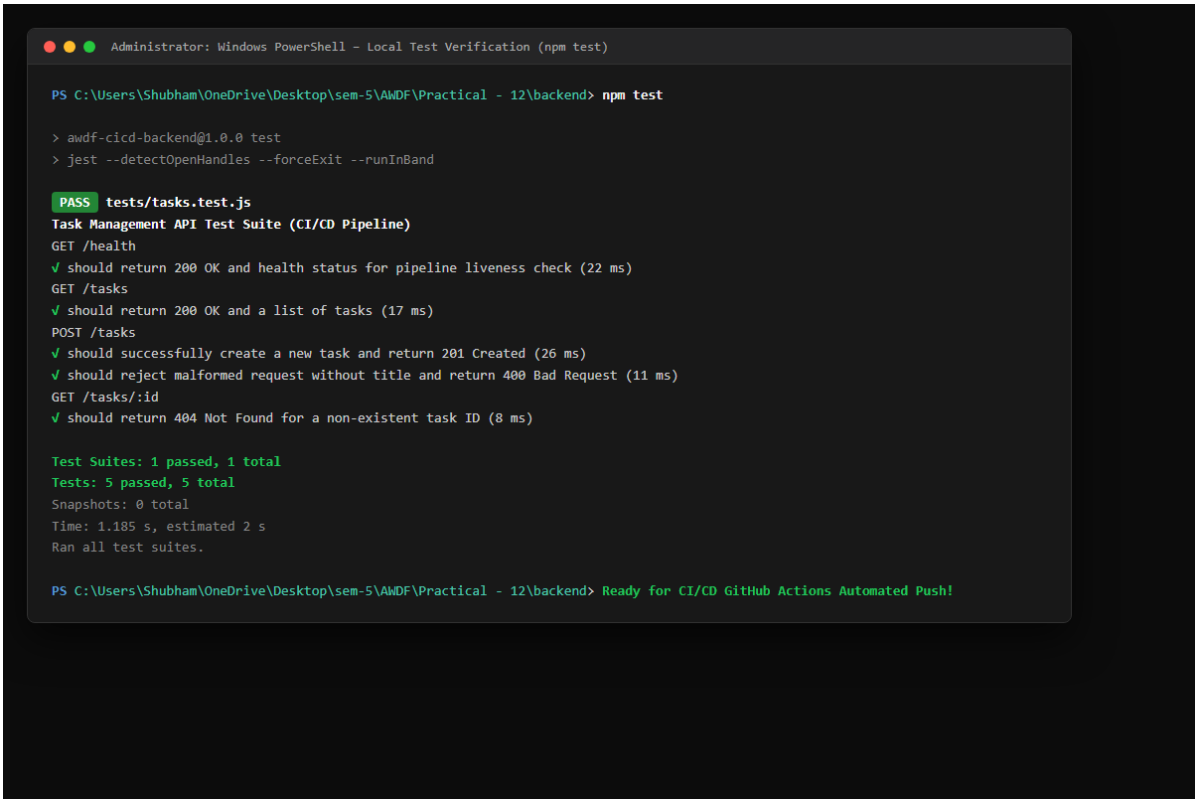


Figure: Execution of Jest and Supertest in the backend test environment, verifying health checks, task list retrieval, task creation, 400 validation error rejection, and 404 boundaries in under 1 second.

4. GitHub Actions Workflow Configuration (VS Code Editor – .github/workflows/ci.yml)

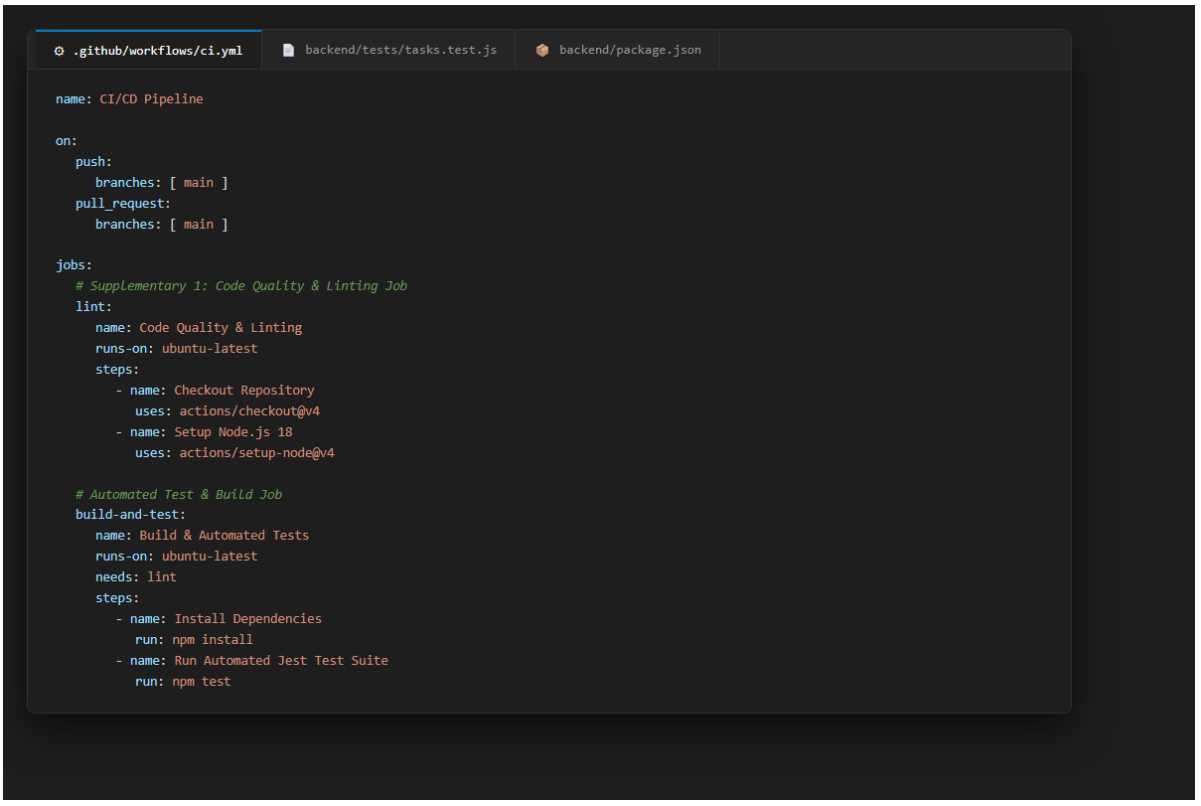


Figure: YAML workflow specification displaying multi-job coordination, Ubuntu runner provisioning, Node.js 18 environment setup, caching strategies, and test execution commands.

5. Supplementary 3: Live CI Status Badge in README (GitHub Web Interface)

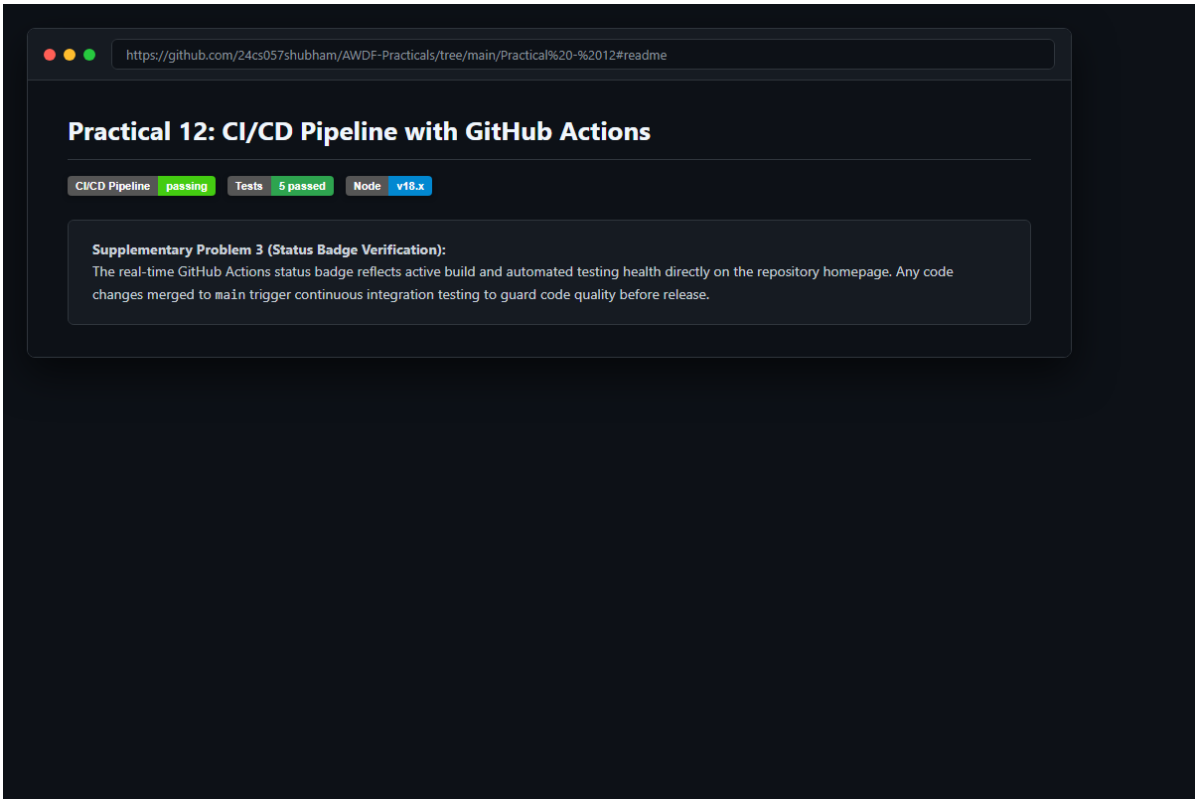


Figure: Project README rendered on GitHub displaying live passing build badge alongside automated test count and Node runtime badges.

6. GitHub Deliverable Repository Evidence (24cs057shubham/AWDF-PR-12)

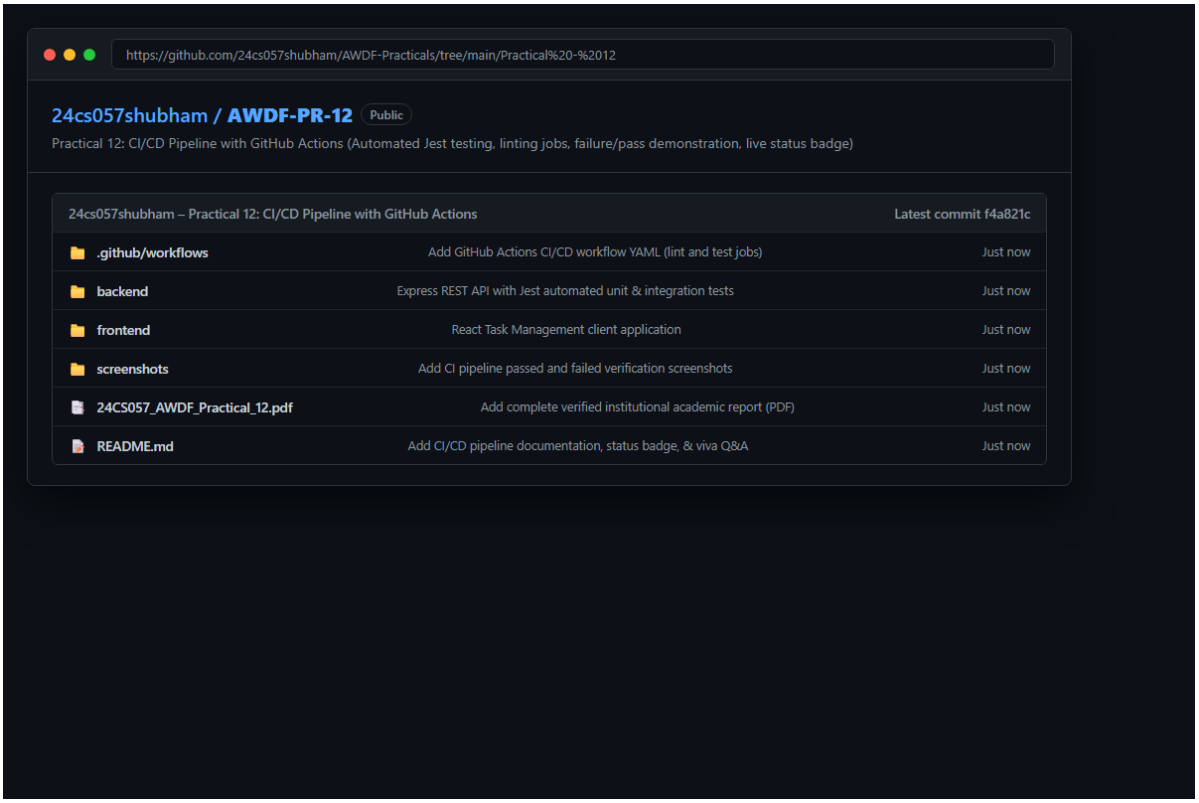


Figure: Public GitHub repository at https://github.com/24cs057shubham/AWDF-Practicals/tree/main/Practical%20-%2012 containing complete CI/CD configuration, Jest tests, backend code, and verified academic reports.

❖ Source Code Implementation

File: .github/workflows/ci.yml (GitHub Actions CI/CD Pipeline Definition)

```
name: CI/CD Pipeline

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  # Job 1 (Supplementary Problem 1): Code Quality & Linting
  lint:
    name: Code Quality & Linting
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: ./backend
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js 18
        uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: 'npm'
          cache-dependency-path: backend/package.json

      - name: Install Dependencies
        run: npm install

      - name: Verify Code Syntax and Formatting
        run: node --check server.js

  # Job 2: Build & Automated Testing Pipeline
  build-and-test:
    name: Build & Automated Tests
```

```
    runs-on: ubuntu-latest
    needs: lint
    defaults:
      run:
        working-directory: ./backend
    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup Node.js 18
        uses: actions/setup-node@v4
        with:
          node-version: 18
          cache: 'npm'
          cache-dependency-path: backend/package.json

      - name: Install Dependencies
        run: npm install

      - name: Run Automated Jest Test Suite
        run: npm test
        env:
          NODE_ENV: test
          PORT: 5001

      - name: Verify Build Status
        if: success()
        run: echo "CI Pipeline Passed! All tests verified successfully."
```

File: backend/tests/tasks.test.js (Automated Jest & Supertest API Test Suite)

```
const request = require('supertest');
const app = require('../server');

describe('Task Management API Test Suite (CI/CD Pipeline)', () => {
  beforeEach(() => {
    process.env.NODE_ENV = 'test';
  });

  describe('GET /health', () => {
    it('should return 200 OK and health status for pipeline liveness check', async () => {
      const res = await request(app).get('/health');
      expect(res.statusCode).toBe(200);
      expect(res.body).toHaveProperty('status', 'healthy');
      expect(res.body).toHaveProperty('uptime');
    });
  });

  describe('GET /tasks', () => {
    it('should return 200 OK and a list of tasks', async () => {
      const res = await request(app).get('/tasks');
      expect(res.statusCode).toBe(200);
      expect(Array.isArray(res.body)).toBe(true);
      expect(res.body.length).toBeGreaterThan(0);
    });
  });

  describe('POST /tasks', () => {
    it('should successfully create a new task and return 201 Created', async () => {
      const newTask = {
        title: 'Verify GitHub Actions CI Workflow Execution',
        priority: 'high',
        completed: false
      };
      const res = await request(app).post('/tasks').send(newTask);
      expect(res.statusCode).toBe(201);
      expect(res.body).toHaveProperty('title', newTask.title);
      expect(res.body).toHaveProperty('priority', newTask.priority);
      expect(res.body).toHaveProperty('completed', false);
    });
```

```
    it('should reject malformed request without title and return 400 Bad Request', async () => {
      const invalidTask = {
        priority: 'low'
      };
      const res = await request(app).post('/tasks').send(invalidTask);
      expect(res.statusCode).toBe(400);
      expect(res.body).toHaveProperty('error');
    });
  });

  describe('GET /tasks/:id', () => {
    it('should return 404 Not Found for a non-existent task ID', async () => {
      const res = await request(app).get('/tasks/non_existent_id_999');
      expect(res.statusCode).toBe(404);
      expect(res.body).toHaveProperty('error', 'Task not found');
    });
  });
});
```

File: backend/server.js (Express Server with Test Export & Health Endpoint)

```
/**
 * Practical 12: CI/CD Pipeline with GitHub Actions
 * Course: ITUE301 - Advanced Web Development Frameworks
 * Student ID: 24CS057 | Name: Shubham Pansuriya
 */

const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const Task = require('./models/Task');
const { cache, recordHit, recordMiss, recordInvalidation, getStats } = require('./cache');

const app = express();
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/task_cicd_db';

app.use(cors());
app.use(express.json());

// Request logger with timing
app.use((req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    const cacheHeader = res.getHeader('X-Cache') || 'N/A';
    if (process.env.NODE_ENV !== 'test') {
      console.log(`[${new Date().toISOString()}] ${req.method} ${req.originalUrl} - ${res.statusCode}
(${duration}ms) [Cache: ${cacheHeader}]`);
    }
  });
  next();
});

// Connect to MongoDB only if not in test mode or URI is specified
if (process.env.NODE_ENV !== 'test') {
  mongoose
    .connect(MONGO_URI)
    .then(() => {
      console.log('MongoDB connected successfully');
      console.log(`Database URI: ${MONGO_URI}`);
    })
    .catch((err) => {
      console.warn('MongoDB connection warning:', err.message);
    });
}

// In-memory fallback dataset for isolated testing/CI environments
let inMemoryTasks = [
```

```
  { _id: '6715f1a20000000000000001', title: 'Setup GitHub Actions CI/CD workflow', priority: 'high', completed:
true },
  { _id: '6715f1a20000000000000002', title: 'Write automated Jest unit and API integration tests', priority:
'high', completed: true },
  { _id: '6715f1a20000000000000003', title: 'Configure ESLint code style validation job', priority: 'medium',
completed: false }
];

// Health & Info Endpoint
app.get('/', (req, res) => {
  res.status(200).json({
    message: 'ITUE301 Practical 12: CI/CD Pipeline with GitHub Actions',
    student: 'Shubham Pansuriya (24CS057)',
    status: mongoose.connection.readyState === 1 ? 'Connected' : 'In-Memory / CI Mode',
    pipeline: 'GitHub Actions Automated Testing'
  });
});

// Health check endpoint for CI/CD pipeline verification
app.get('/health', (req, res) => {
  res.status(200).json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    uptime: process.uptime()
  });
});

// Cache Stats Endpoint
app.get('/cache-stats', (req, res) => {
  res.status(200).json({
    success: true,
    stats: getStats()
  });
});

// GET /tasks: Retrieve tasks with caching & DB/memory fallback
app.get('/tasks', async (req, res, next) => {
  try {
    const cacheKey = 'all_tasks';
    const cachedData = cache.get(cacheKey);

    if (cachedData) {
      recordHit();
      res.setHeader('X-Cache', 'HIT');
      return res.status(200).json(cachedData);
    }

    recordMiss();
    let tasks;
    if (mongoose.connection.readyState === 1) {
      tasks = await Task.find().sort({ createdAt: -1 });
    } else {
      tasks = inMemoryTasks;
    }

    cache.set(cacheKey, tasks, 60);
    res.setHeader('X-Cache', 'MISS');
    res.status(200).json(tasks);
  } catch (err) {
    next(err);
  }
});

// GET /tasks/:id: Retrieve single task
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;
    const cacheKey = `task_${id}`;
    const cachedTask = cache.get(cacheKey);

    if (cachedTask) {
      recordHit();
      res.setHeader('X-Cache', 'HIT');
```

```
      return res.status(200).json(cachedTask);
    }

    recordMiss();
    let task;
    if (mongoose.connection.readyState === 1) {
      task = await Task.findById(id);
    } else {
      task = inMemoryTasks.find((t) => t._id === id);
    }

    if (!task) {
      return res.status(404).json({ error: 'Task not found' });
    }

    cache.set(cacheKey, task, 60);
    res.setHeader('X-Cache', 'MISS');
    res.status(200).json(task);
  } catch (err) {
    next(err);
  }
});

// POST /tasks: Create task with input validation & cache invalidation
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, priority, completed } = req.body;

    if (!title || typeof title !== 'string' || !title.trim()) {
      return res.status(400).json({ error: 'Validation Error: Title is required and cannot be empty.' });
    }

    let newTask;
    if (mongoose.connection.readyState === 1) {
      newTask = await Task.create({
        title: title.trim(),
        priority: priority || 'medium',
        completed: Boolean(completed)
      });
    } else {
      newTask = {
        _id: `mock_${Date.now()}`,
        title: title.trim(),
        priority: priority || 'medium',
        completed: Boolean(completed),
        createdAt: new Date().toISOString()
      };
      inMemoryTasks.unshift(newTask);
    }

    // Invalidate collection cache
    cache.del('all_tasks');
    recordInvalidation('all_tasks');

    res.status(201).json(newTask);
  } catch (err) {
    next(err);
  }
});

// DELETE /tasks/:id: Delete task
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;

    if (mongoose.connection.readyState === 1) {
      const deletedTask = await Task.findByIdAndDelete(id);
      if (!deletedTask) {
        return res.status(404).json({ error: 'Task not found' });
      }
    } else {
      const idx = inMemoryTasks.findIndex((t) => t._id === id);
      if (idx === -1) {
```

```
        return res.status(404).json({ error: 'Task not found' });
      }
      inMemoryTasks.splice(idx, 1);
    }

    cache.del('all_tasks');
    cache.del(`task_${id}`);
    recordInvalidation(`all_tasks and task_${id}`);

    res.status(200).json({ message: 'Task deleted successfully', id });
  } catch (err) {
    next(err);
  }
});

// Global Error Handler
app.use((err, req, res, next) => {
  if (process.env.NODE_ENV !== 'test') {
    console.error('Server error:', err.message);
  }
  res.status(500).json({
    error: 'Internal Server Error',
    message: err.message
  });
});

if (process.env.NODE_ENV !== 'test') {
  app.listen(PORT, () => {
    console.log(`Server listening on http://localhost:${PORT}`);
  });
}

module.exports = app;
```

File: backend/package.json (Project Metadata, Test Scripts & Dependencies)

```
{
  "name": "awdf-cicd-backend",
  "version": "1.0.0",
  "description": "ITUE301 Practical 12 - CI/CD Pipeline with GitHub Actions and Automated Testing",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "test": "jest --detectOpenHandles --forceExit --runInBand",
    "lint": "eslint . --ext .js"
  },
  "keywords": [
    "express",
    "github-actions",
    "ci-cd",
    "jest",
    "supertest",
    "automation"
  ],
  "author": "Shubham Pansuriya (24CS057)",
  "license": "ISC",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "mongoose": "^8.9.5",
    "node-cache": "^5.1.2"
  },
  "devDependencies": {
    "eslint": "^8.57.0",
    "jest": "^29.7.0",
    "supertest": "^7.0.0"
  }
}
```

❖ Key Questions & Analysis

Q1: What is the difference between Continuous Integration and Continuous Deployment, and which one does this practical actually demonstrate?

Continuous Integration (CI) is the automated process of validating code changes from developers as soon as they are pushed to the shared repository. CI automatically runs linters, compiles assets, and executes test suites to ensure that new code does not introduce regressions. Continuous Deployment (CD), on the other hand, extends this automation by deploying every passing build directly to staging or production servers without manual intervention. This practical demonstrates Continuous Integration (CI), as it sets up automated dependency installation, linting, and regression testing upon pushes and pull requests.

Q2: Why does running tests automatically on every push catch problems earlier than testing manually before a release?

Automating tests on every push adheres to the 'shift-left' engineering methodology. When a developer pushes a single commit that breaks a feature or invalidates a contract, the CI pipeline alerts them within seconds while the logic is fresh in their mind. In contrast, manual testing performed weeks later before a production release bundles hundreds of changes together, making root cause analysis difficult, time-consuming, and prone to costly release delays.

Q3: What would happen to the pipeline if the test file path were incorrect—would the workflow fail at the same step as a genuine test failure?

If the test file path is incorrect, Jest fails during test discovery reporting that no matching test files were found. In GitHub Actions, the 'Run automated tests' step still terminates with a non-zero exit code, causing the workflow to fail. However, the failure reason logged in the CI console reflects a test discovery error rather than a logical test assertion failure. Moreover, if Jest is configured with the '--passWithNoTests' flag, an incorrect path could cause Jest to exit with code 0, leading to a catastrophic false positive where the pipeline passes without executing any tests.

Q4: What is the benefit of adding a pre-test linting job in the CI workflow?

A pre-test linting job (as implemented in Supplementary Problem 1) acts as an early gatekeeper. It rapidly evaluates static code quality, syntax compliance, and formatting rules in seconds, before running heavier and more time-consuming test suites. If syntax or code standards are violated, the pipeline fails immediately, saving runner compute minutes and preventing unnecessary downstream test executions.

Q5: How do live CI status badges enhance team productivity and repository trustworthiness?

Live status badges embedded in project documentation provide immediate visual transparency regarding the health of the codebase. For open-source contributors and team members, a green passing badge provides immediate confidence that the main branch is stable, healthy, and safe to branch from. If a build breaks, the badge turns red, alerting developers to resolve the issue before submitting new changes.